TÉCNICO
UNIVERSIDADE
DE LISBOA

Autonomous Systems

# Decision Making for Mobile Robot Navigation using AlphaBot2

Group 18
Presentation 2

David Gaspar - 106541
Francisco Valério - 106533
Miguel Carneiro - 106218
Pedro Mónico - 106626

U LISBOA | UNIVERSIDADE DE LISBOA

# OVERVIEW

## SENSORS TESTING

Sensor data acquisition
Characterization of drift and uncertainties
Understanding real-world limitations

## MDP & RL FORMULATION

Formalization of robot decision problem, Definition of MDP
Reinforcement Learning approach, Model-free learning, Exploration vs exploitation trade-off

## INITIAL IMPLEMENTATION: MICRO - SIMULATOR

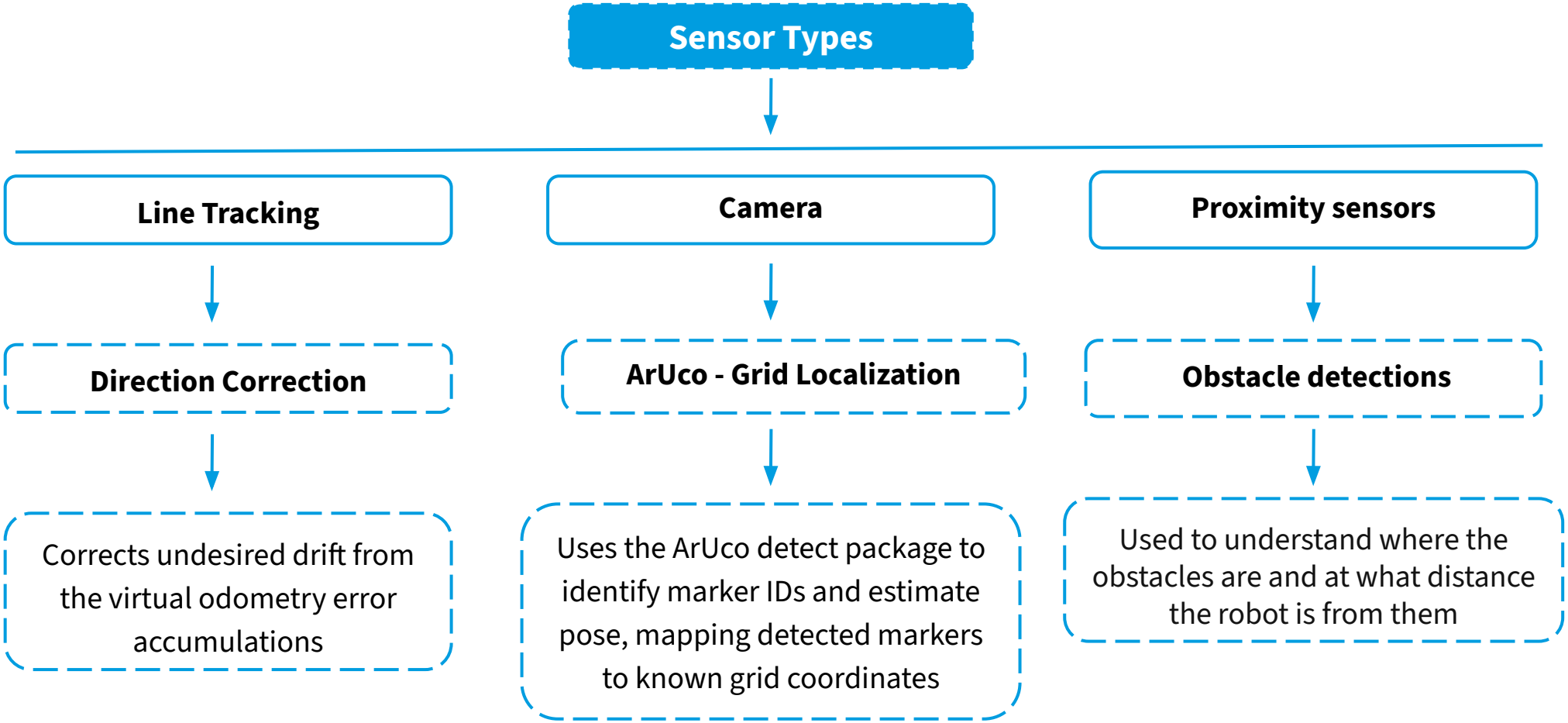
First skeleton of MDP algorithm
Integration of motion + reward logic

## SIMULATION ENVIRONMENT

Gazebo virtual model of Alphabot2 robot

## NEXT STEPS OF WORKPLAN

# SENSORS TESTING


Sensor Types
Line Tracking
Direction Correction
Corrects undesired drift from the virtual odometry error accumulations
Camera
ArUco - Grid Localization
Uses the ArUco detect package to identify marker IDs and estimate pose, mapping detected markers to known grid coordinates
Proximity sensors
Obstacle detections
Used to understand where the obstacles are and at what distance the robot is from them

**Sensor Data**

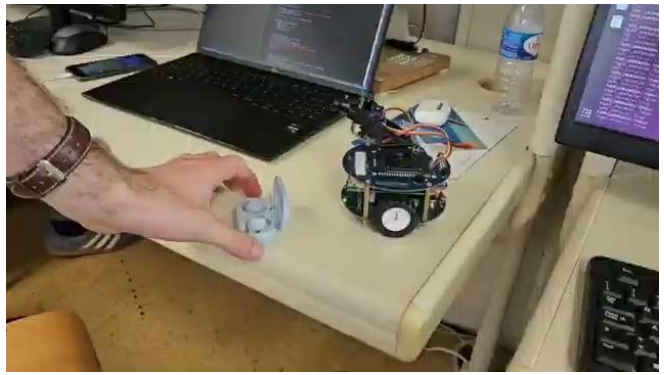
**Motion**

Undesired rigth/left drift in the transational movement

Rotation movement with no fixed axis

**No precise position estimate**

**Proximity sensors**



**Camera**

**Latency problems**

**Motion blur**

**Narrow Viewing angle**

# MDP & RL FORMULATION

A **Markov Decision Process (MDP)** is a formal mathematical model for decision making under uncertainty, where outcomes are partly random and partly controlled by the agent.

**It is defined by**

- S → Set of possible **states** the robot can be in (position, angular orientation, velocity, acceleration)
- A → Set of **actions** available to the robot (foward, backwards, right, left)
- T($s$, $a$, $s'$) → **Transition probability** of reaching s' from s after action a
- R($s$, $a$) → **Reward** received for taking action a in state s
- $\gamma \in [0,1]$ → **Discount factor**, balancing immediate ($\gamma = 0$) vs future ($\gamma = 1$) rewards

The future depends only on the **current state and action** → $p(s_{t+1} \mid s_t, a_t)$

A **policy** $\pi$ maps states to actions telling the robot what is **the most optimal to do in every single situation**.

To find **the optimal policy** $\pi^*$, we first compute iteratively the value function **V*(s)** (a measure of how good it is to be in state s assuming optimal behavior from that point onward) which is defined recursively by the **Bellman Equation**:

$$V^*(s) = \max_a \left[ R(s,a) + \gamma \sum_{s'} T(s,a,s') V^*(s') \right]$$

There are 2 classical approaches to solve it:

- **Value Iteration**: repeatedly update the value of every state until convergence;
- **Policy Iteration**: start with any policy, evaluate it, improve it and repeat until optimal.

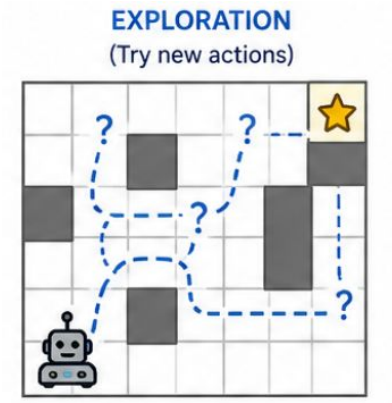
**PROBLEM: TRANSITION PROBABILITY T(s, a, s') IS UNKOWN AND NOT DETERMINISTIC**
**SOLUTION: REINFORCEMENT LEARNING APPROACH**

An agent learns to make decisions by interacting with an environment since is **has not prior knowledge** of it.

Unlike in MDPs, the policy must be **learned from experience** rather than computed from a **known model**.

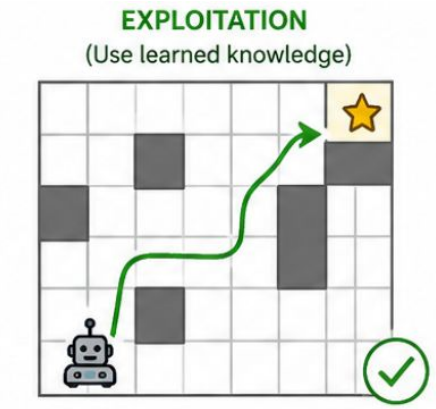
**Reinforcement Learning** →

- When **transition model** T(*s, a, s'*) is **unknown**
- Robot learns optimal behavior through **trial and error**
- Action selection is driven by Exploration & Exploitation trade-off




Try new actions to discover better strategies

Use current best-known actions to maximize reward

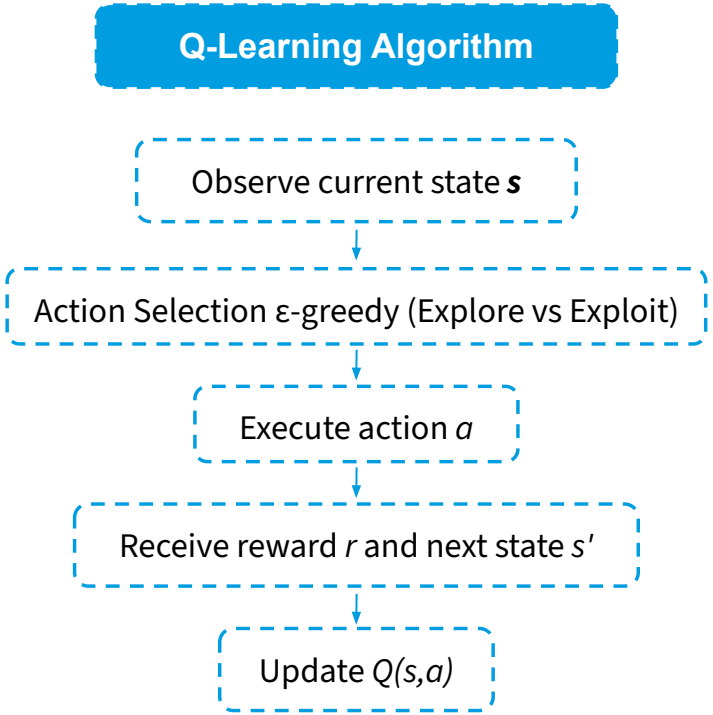


**Q-Learning Algorithm** →

- Learns action-value function ***Q(s,a)*** **directly from experience**
- Updates Q-values based on **observed rewards and transitions**
- Converges to **optimal policy *Q**** given sufficient exploration

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

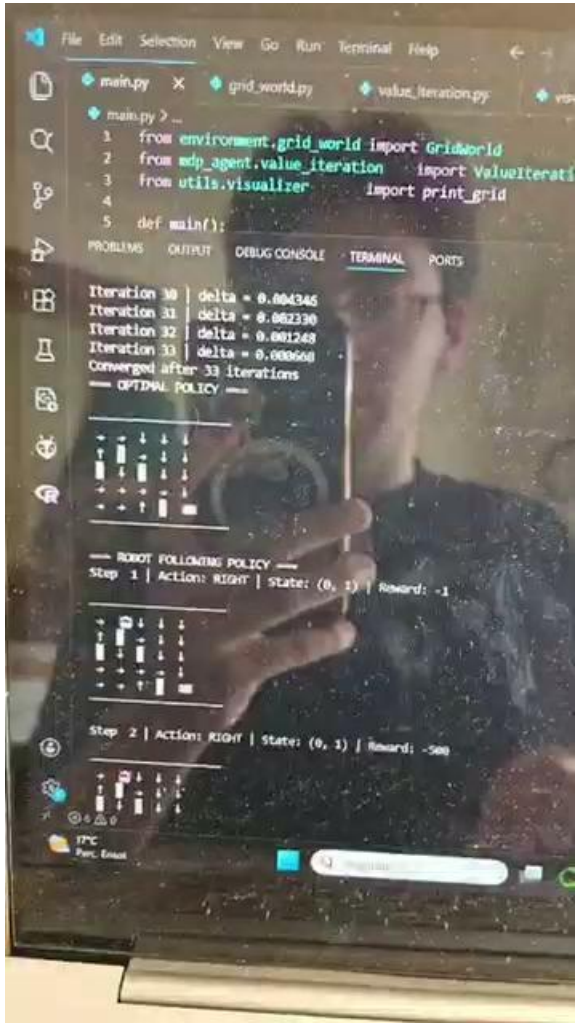
| $Q(s,a)$ | $\alpha$ | $r$ | $\gamma$ | $\max_{a'} Q(s',a')$ | $Q(s',a')$ |
|---|---|---|---|---|---|
| Current value | Learning rate $(0 < \alpha \leq 1)$ | Reward received | Discount factor $(0 \leq \gamma < 1)$ | Best future value | Value of next state–action |



# INITIAL IMPLEMENTATION: MICRO - SIMULATOR

# INITIAL IMPLEMENTATION: MICRO - SIMULATOR

## First skeleton of MDP algorithm



# SIMULATION ENVIRONMENT

# Simulation Environment

## Gazebo virtual model of Alphabot 2 robot

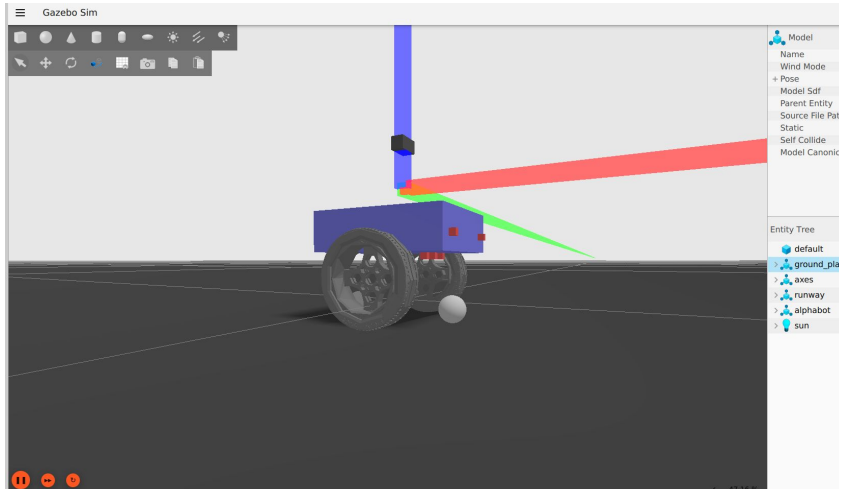
**Gazebo Alphabot2 Model Adaptation**

- **IR - Sensors**
- **Camera**
- **Line Tracking - Sensors**

**ROS2 and Gazebo (topic/subscriber interface)**

**A more realistic micro-simulator that takes into account real time sensor data**

**Still presents limitations regarding camera and odometry in comparison with the real scenario**



# NEXT STEPS OF WORKPLAN

# NEXT STEPS OF WORKPLAN

| TASK | DESCRIPTION |
| --- | --- |
| **Complete initial MDP & RL algorithm** | Finalize first working version of state definition, action space and reward function; Implement Q-learning loop |
| **Implement grid world in Gazebo** | Create structured navigation environment; Define obstacles,goal positions and free space zones |
| **Collect real sensor data (ROS bag recording)** | Record real sensor data and feed it on the algorithm to test state estimation, RL decisions and sensor fusion logic |

# References

[1] T. Babu, R. R. Nair, V. S., D. H. R., and N. M., “Markov Decision Processes in Autonomous Robot Navigation,” in Proc. <Conference Name>, 2024.

[2] J. Girard and M. R. Emami, “Concurrent Markov decision processes for robot team learning,” Engineering Applications of Artificial Intelligence, vol. 39, pp. 225–234, 2015.

# Thank You!